

National University of Computer and Emerging Sciences, Lahore Campus

Course: Data Structures Course: CS 2001 Program: BS(CS) Semester: Fall 2024

Total marks: 50, Due Date: 8th Dec '24

Type: Assignment 5 Page(s): 8

PROBLEM BACKGROUND

Question 1: (Graphs) [Marks :35]

During the Pandemic CoVID, the classes and labs in universities are mostly conducted online. The load on the internet service and Local area network (LAN) has significantly increased. It is observed in NUCES that our LAN is not properly designed and it is not using the available resources to its fullest. As you people are familiar with the fact that the NUCES Lahore campus consists of various buildings so the LAN needs to connect the classrooms, labs, faculty offices in these buildings efficiently.

We have modeled this problem to a graph, where computers in classrooms, labs and offices are represented by nodes and the networking wires that connect them are represented by edges. The edges have weight that indicates the length of the wire needed to connect two nodes. Your task is to take this graph as input and design a LAN network, FLAN that covers all the nodes with minimum wire length and do so efficiently.

Input
1. Input a **weighted undirected graph** G(V, E). 2. A vertex v in V represents a computer in NUCES. Each computer has a unique ID and location (office, lab or classroom). As it is a weighted graph so each vertex has a weight that is initialized to a max int it can hold. 3. The edge has a weight that indicates the length of the wire needed to connect two computers.

Your task is to take a graph as input and design a LAN network, FLAN that covers all the nodes and do so efficiently using following idea

1. Input graph G(V,E), in an adjacency list representation from a file 2. Select a vertex v of Graph G randomly and sets its weight = 0

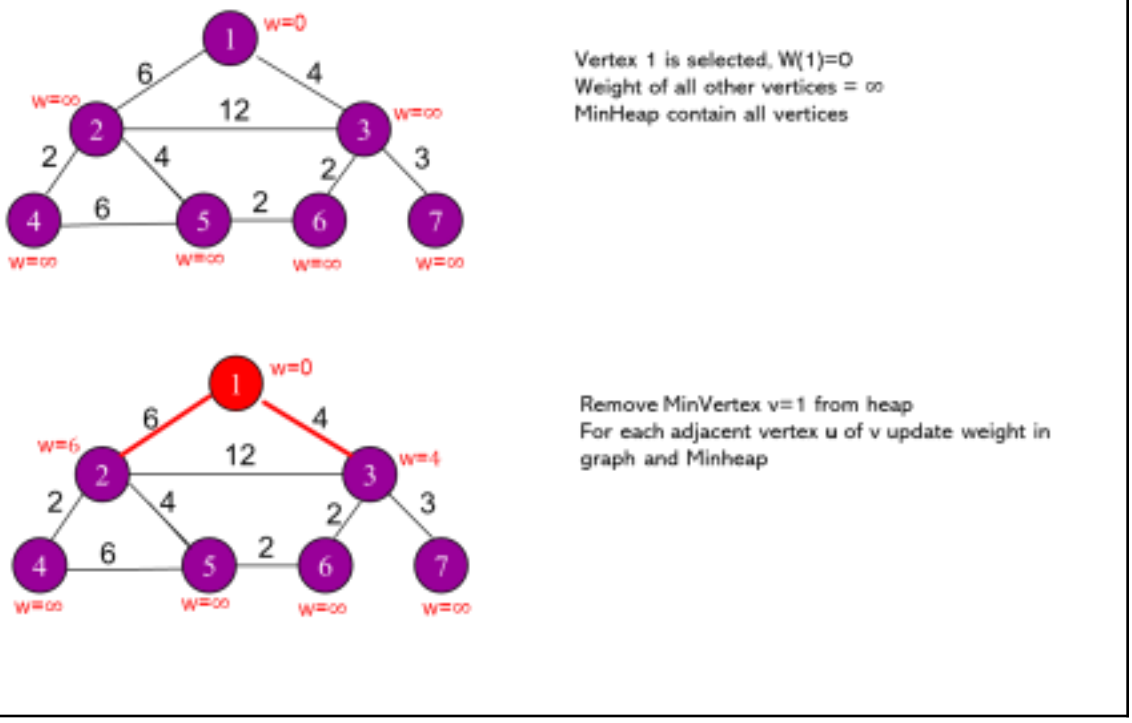
3. Build a MinHeap on V,vertices using weight of the vertices as key in heap. 4. While MinHeap is not empty, do

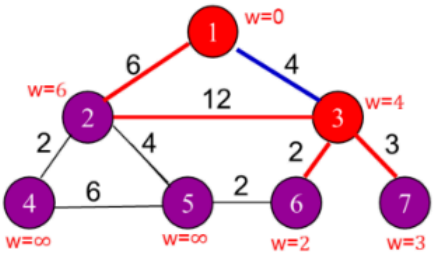
a. ExtractMin vertex v from MinHeap

b. For each adjacent vertex **u** of v

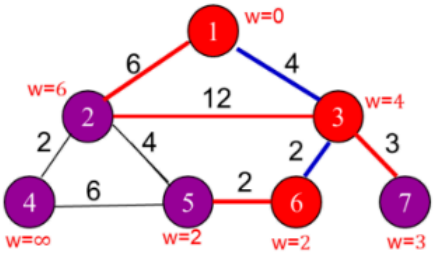
i. If **u** is in MinHeap and its weight is more than weight of edge (v,u), then update the weight of u with weight of edge (v,u) by using decreaseKey function in MinHeap so that its position in the heap is properly updated.

Example:

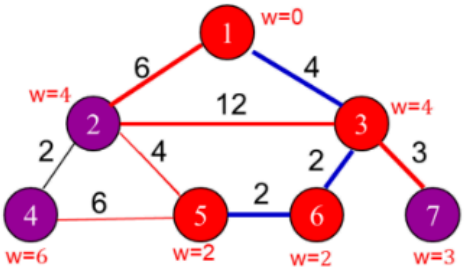




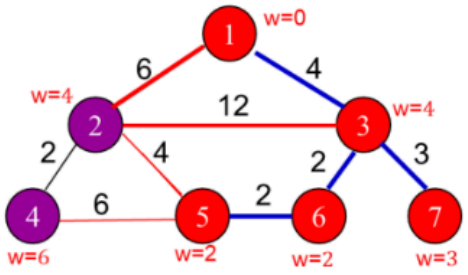
Remove Min weight Vertex v from heap
For each adjacent vertex **u** of v=3 in minheap update weight if necessary



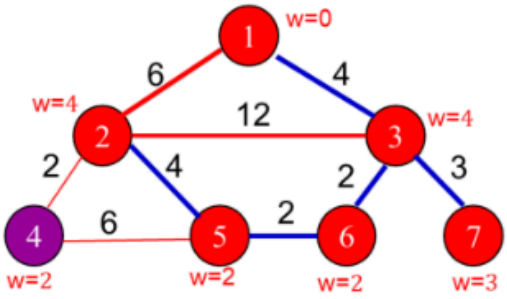
Extract Min weight Vertex v from heap
For each adjacent vertex **u** of v=6 in minheap update weight if necessary



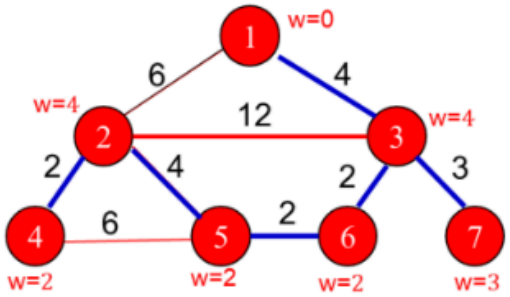
Extract Min weight Vertex v from heap
For each adjacent vertex **u** of v=5 in minheap update weight if necessary



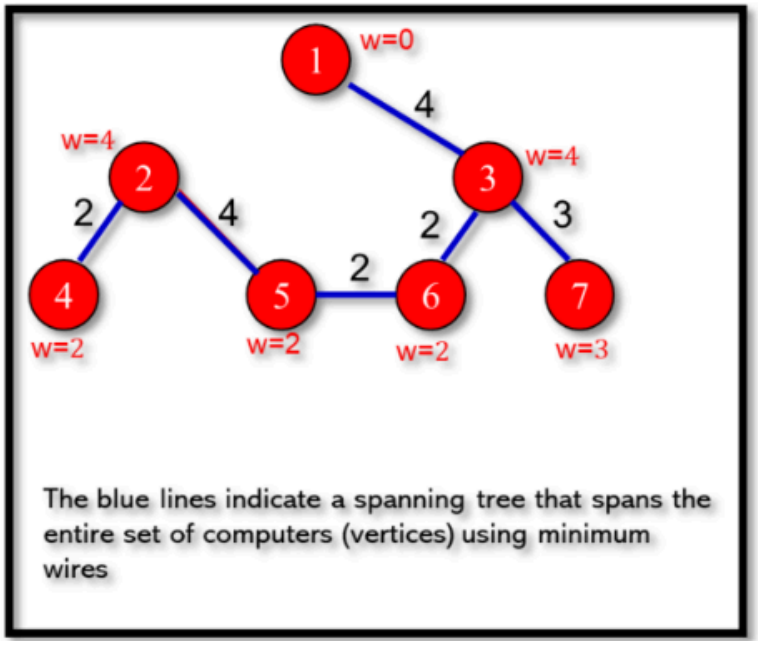
Extract Min weight Vertex v from heap
For each adjacent vertex **u** of v=7 in minheap update weight if necessary



Extract Min weight Vertex v from heap
For each adjacent vertex u of v=2 in minheap update weight if necessary



Extract Min weight Vertex v from heap
For each adjacent vertex u of v=4 in minheap update weight if necessary



Input File Format

Total number of vertices
Vertex info in the form (vertex ID, type)
Total number of edges
edge info (starting vertex id, ending vertex id), weight

7
1, office1
2, lab1
3, lab 4
4, lab 9
5, lab 10
6, office2
7, office6
9
1,3 4
1,2 6
2,4 2
2,5 4
2,3 12
3,6 2
3,7 3
4,5 6
5,6 2

Assignment Requirements

Create a class for graph node
Create a class for graph
Use adjacency list representation to maintain edge information
Use STD list for adjacency list representation

Build a Minheap class with functions

1. Constructor
2. Destructor
3. BuildHeap
4. ExtractMin
5. DecreaseKey

Keep information about the position of each vertex in the Heap. You can use an array of size |V| and store the index of the place where vertex exists in the heap

Create a Menu in the main for the following functionality 1. Input a graph from a file
2. Save a graph in a file
3. Add a vertex in existing graph G

4. Add an edge in existing graph G
5. Print the set of edges that covers all the vertices using the above algorithm (for above example you will print the blue edges.)

Question 2: (HashMap with Linear Probing) [Marks :15]

Implement a HashItem struct which represents an item in a hash array.

```
template <class v>
struct HashItem
{
int key;
v value;
short status;

};
```

status variable can have 0, 1 or 2. 0 means empty, 1 means deleted, 2 means occupied. Status variable will be used by get and delete methods of HashMaps implemented in the next questions. The default value assigned to a HashItem is 0 (empty).

Now implement a HashMap class whose basic definition is as follows (You can add any helping member variables and methods):

```
template <class v>
class HashMap
{
private:
HashItem<v>* hashArray;
int capacity;
int currentElements;
virtual int getNextCandidateIndex(int key, int i)

public:
HashMap();
HashMap(int const capacity);
void insert(int const key, v const value);
bool deleteKey(k const key) const;
v* get(k const key) const;
~HashMap();
};
```

1. HashMap(): constructor assigns a capacity of 10 to hashArray.

2. HashMap(int const capacity): an overloaded constructor that assigns the

capacity of given capacity to hashArray. If capacity is less than 1 return error via assert(capacity>1)

3. void insert(int const key, v const value):
a. The insert method inserts the value at its appropriate location. Find the first candidate index of the key using:

index= key ***mod*** *capacity*

b. To resolve hash collision, it will use the function getNextCandidateIndex(key, i) to get the next candidate index. If the candidate index also has collision, then getNextCandidateIndex will be called again with an increment in i. getNextCandidateIndex will be continued to call until we find a valid index. Initially i will be 1.

c. If the loadFactor becomes 0.75, then it will call the doubleCapacity method to double the capacity of array and rehash the existing items into the new array.

4. void doubleCapacity(): A private method which doubles the capacity of hash array and rehashes the existing items. Use getNextCandidateIndex method to resolve collision.

5. virtual int getNextCandidateIndex(int key, int i): a private and virtual method that uses linear probing to return the next candidate index for storing the item containing key k. Linear probing means that it will simply add i to the hash value of key. This method does not check whether the candidate index has collision or not.

6. bool deleteKey(k const key) const: this method deletes the given key. It returns true if the key was found. If the key was not found it returns false. When the key is found, simply set the status of the hashitem containing the key to deleted (value of 1). It also uses status variable to search for the key intelligently.

7. V* get(k const key) const: this method returns a pointer to the corresponding value of the key. If the key is not found, it returns nullptr. It also uses status variable to search for the key intelligently.

8. ~HashMap(): destructor

Also implement the following.

· Create a class **QHashMap** which inherits the HashMap class implemented above. Override the getNextCandidateIndex(int key, int i) method so that it performs quadratic probing, i.e., add the square of i to the hash value of key.

· Create a class **DHashMap** which inherits the HashMap class implemented above. Override the getNextCandidateIndex(int key, int i) method so that it performs double hashing and returns the candidate index. Double hashing will be performed as follows:

first_value= key ***mod*** *capacity*

second_value= (PRIME - (key ***mod*** *PRIME))* (PRIME is any prime number.) *candidate index= (first_value + i*second_value)* ***mod*** *capacity*

· Create a global function populateHash which is passed a filename as parameter and a HashMap object by pointer void populateHash(string filename, HashMap<string> *hash). The function reads <id, name> pairs and populates the hash with those pairs. The key is id.

Now run the following main program:

```
#include <iostream>
using namespace std;
#include <string>

int main()
{

HashMap<string> *map;
map=new HashMap<string>;
populateHash("students.txt", map);
cout<<*map->get(9);
map->deleteKey(9);
assert(map->get(9)==nullptr);
delete map;
map=new QHashMap<string>;
populateHash("students.txt", map);
cout<<*map->get(98);
map->deleteKey(98);
assert(map->get(98)==nullptr);
delete map;

map=new DHashMap<string>;
populateHash("students.txt", map);
cout<<*map->get(101);
map->deleteKey(101);
assert(map->get(101)==nullptr); delete
map;

return 0;

}
```